

VITyarthi – Build Your Own Project

SECURE PASSWORD GENERATOR

PROJECT REPORT SUBMISSION

Student Name	Jonnada Nithish
Roll Number	26BCY10115
Branch	CSE (Cyber Security)
Faculty Name	Santhosh Kumar

Submission Report

1. Introduction

The Secure Password Generator is a simple console-based Python project designed to generate random passwords according to requirements selected by the user. It demonstrates functions, modules, input validation, string handling, random generation, conditional logic, and file handling.

The project intentionally avoids a graphical interface, database, external packages, and object-oriented programming. Five meaningful Python modules are used so that responsibilities are clearly separated.

2. Problem Statement

Users sometimes create short, predictable, or repetitive passwords. A simple password-generation tool can help users create random passwords using a chosen length and combination of lowercase letters, uppercase letters, numbers, and symbols.

The project addresses this through a console application that accepts password requirements, generates a random password, provides basic strength feedback, and optionally saves it to a local text file.

3. Functional Requirements

ID	Functional Module	Description
FR1	Password Configuration	Accept length from 8–128 and character-type choices.
FR2	Password Generation	Generate a random password using selected character types.
FR3	Password Analysis & Saving	Give basic strength feedback and optionally save the password.

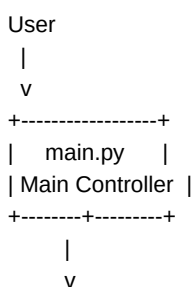
- The user can generate another password or exit.
- At least one character category must be selected.
- Invalid password lengths are rejected and requested again.

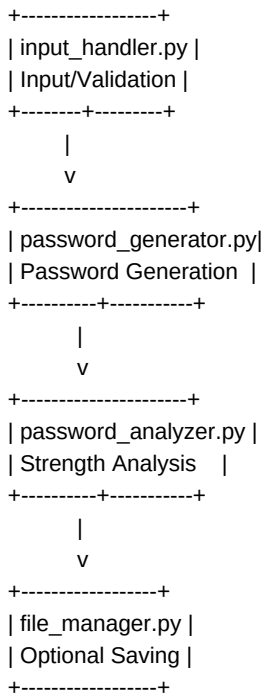
4. Non-Functional Requirements

- Usability — simple console prompts and clear messages.
- Security — Python's secrets module is used for password generation.
- Reliability — invalid input is handled without crashing the program.
- Maintainability — five modules separate major responsibilities.
- Error Handling — invalid numeric input and invalid selections are checked.
- Resource Efficiency — only lightweight standard-library operations are required.

5. System Architecture

The project uses a simple procedural modular architecture. `main.py` coordinates the workflow while each supporting module performs one focused responsibility.

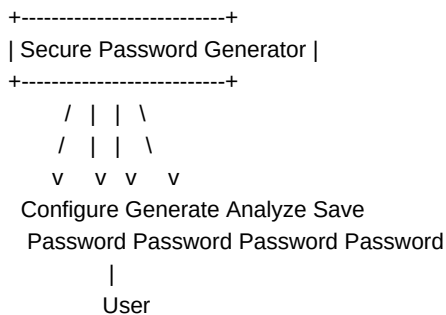




Project files: main.py, input_handler.py, password_generator.py, password_analyzer.py, and file_manager.py.

6. Design Diagrams

6.1 Use Case Diagram



Actor: User. The user configures requirements, generates a password, analyzes it, and optionally saves it.

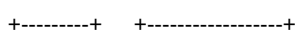
6.2 Workflow Diagram

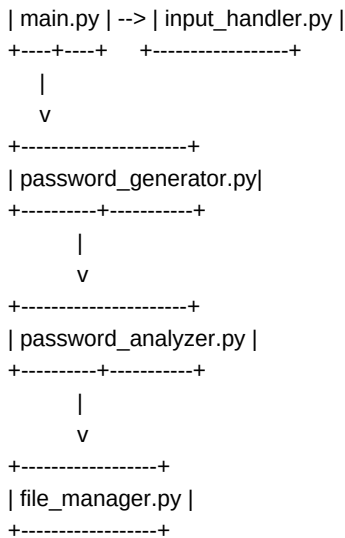
Start -> Enter length -> Validate -> Choose character types
-> Validate selection -> Generate password -> Analyze strength
-> Save if requested -> Generate another? -> End

6.3 Sequence Diagram

User -> main.py: Enter requirements
main.py -> input_handler.py: Validate
input_handler.py -> main.py: Valid choices
main.py -> password_generator.py: Generate
password_generator.py -> main.py: Password
main.py -> password_analyzer.py: Analyze
password_analyzer.py -> main.py: Result
main.py -> file_manager.py: Save if requested
file_manager.py -> main.py: Save result
main.py -> User: Display output

6.4 Component / Module Diagram





A class diagram is not used because the implementation is procedural and does not use classes. An ER diagram is not required because there is no database.

7. Design Decisions & Rationale

Python: Simple, readable, and suitable for a console project.

Console interface: Avoids unnecessary GUI complexity.

Procedural design: Keeps the implementation simple and avoids OOP.

Five modules: Separates responsibilities while meeting the modular implementation requirement.

secrets module: Provides secure random selection for password generation.

No database: The project has no need for structured persistent data.

8. Implementation Details

Input handling: Validates length 8–128 and requires at least one character category.

Password generation: Includes at least one character from each selected category, fills the remaining length, and shuffles the result.

Password analysis: Checks length and character categories and returns Strong, Medium, or Weak with suggestions.

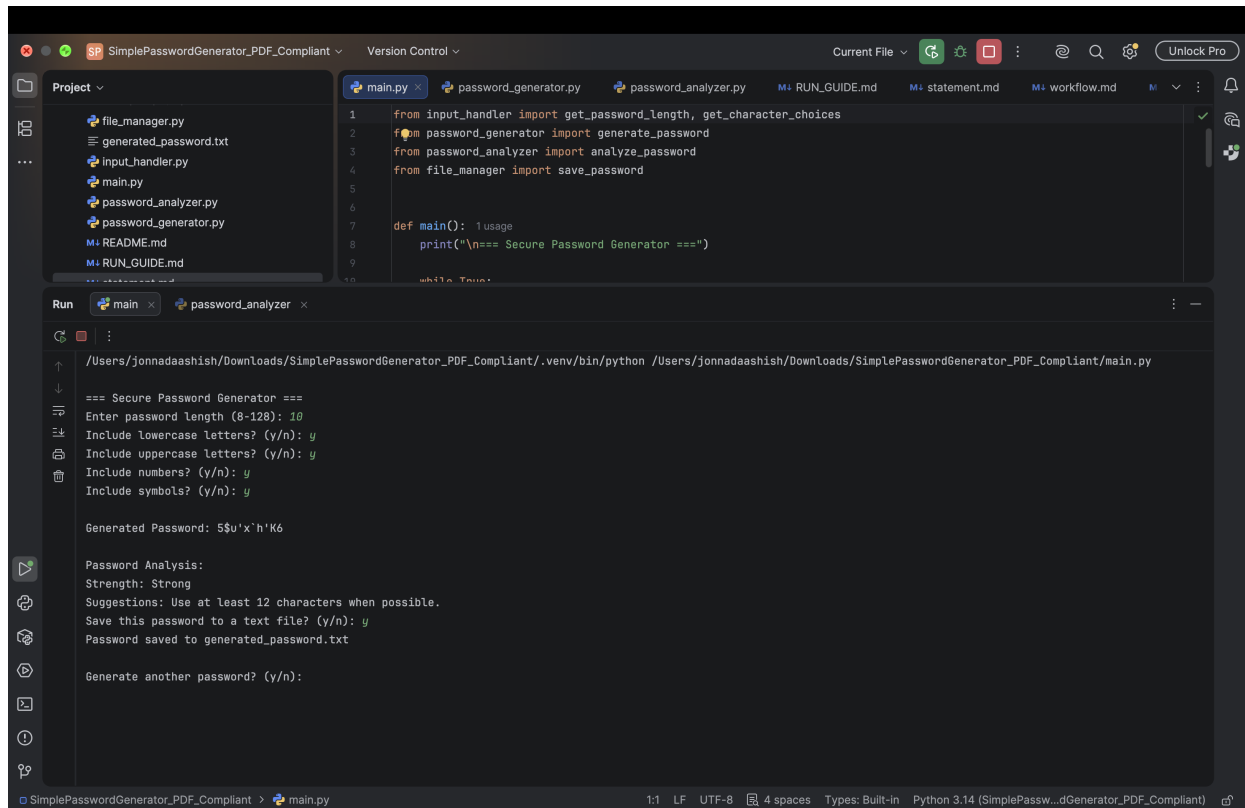
File handling: Optionally writes the generated password to generated_password.txt.

Dependencies: Only Python standard-library modules are used: secrets, string, and pathlib.

9. Screenshots / Results

The following screenshots are the actual console executions provided for the project. They demonstrate password generation, input validation, and password saving behavior.

Figure 1. Successful password generation and password analysis.



```
1 from input_handler import get_password_length, get_character_choices
2 from password_generator import generate_password
3 from password_analyzer import analyze_password
4 from file_manager import save_password
5
6
7 def main(): 1 usage
8     print("\n=== Secure Password Generator ===")
9
10    while True:
```

```
/Users/jonnadaashish/Downloads/SimplePasswordGenerator_PDF_Compliant/.venv/bin/python /Users/jonnadaashish/Downloads/SimplePasswordGenerator_PDF_Compliant/main.py

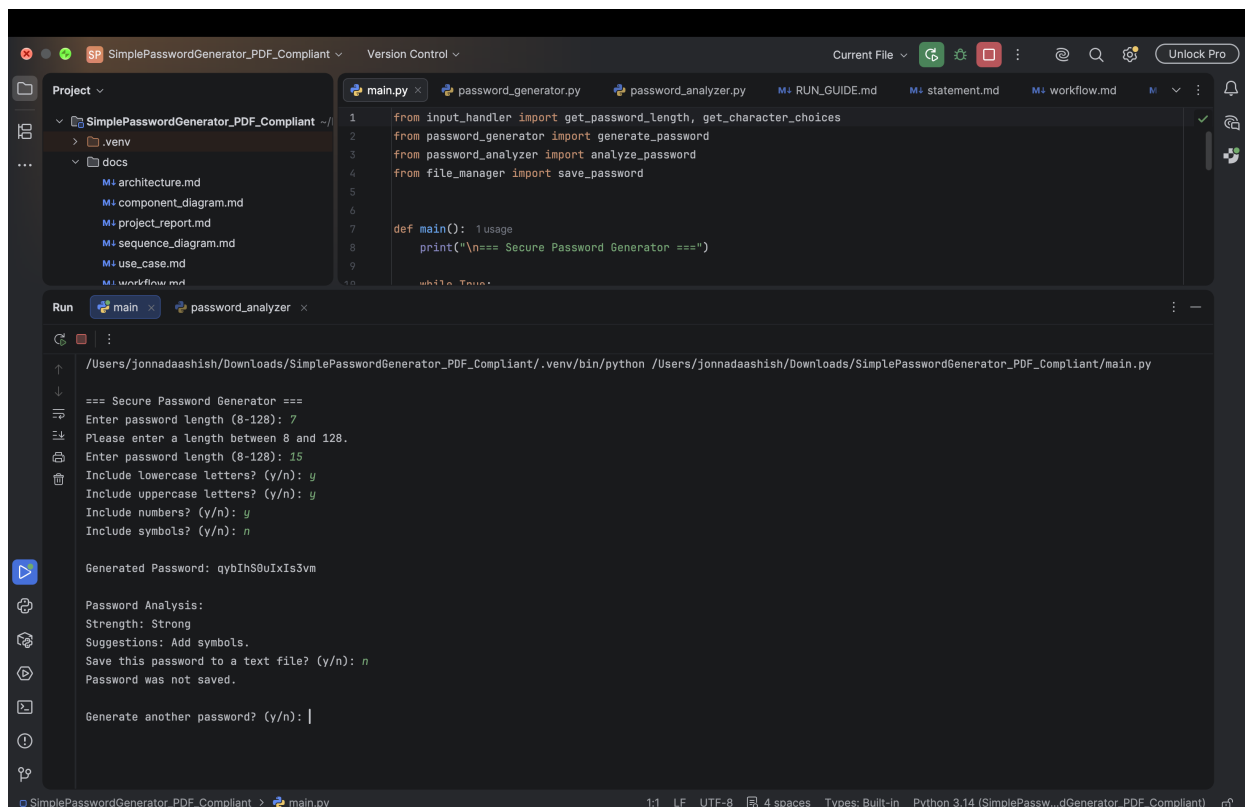
=== Secure Password Generator ===
Enter password length (8-128): 10
Include lowercase letters? (y/n): y
Include uppercase letters? (y/n): y
Include numbers? (y/n): y
Include symbols? (y/n): y

Generated Password: 5$u'x'h'K6

Password Analysis:
Strength: Strong
Suggestions: Use at least 12 characters when possible.
Save this password to a text file? (y/n): y
Password saved to generated_password.txt

Generate another password? (y/n):
```

Figure 2. Input validation when an invalid password length is entered.



```
1 from input_handler import get_password_length, get_character_choices
2 from password_generator import generate_password
3 from password_analyzer import analyze_password
4 from file_manager import save_password
5
6
7 def main(): 1 usage
8     print("\n=== Secure Password Generator ===")
9
10    while True:
```

```
/Users/jonnadaashish/Downloads/SimplePasswordGenerator_PDF_Compliant/.venv/bin/python /Users/jonnadaashish/Downloads/SimplePasswordGenerator_PDF_Compliant/main.py

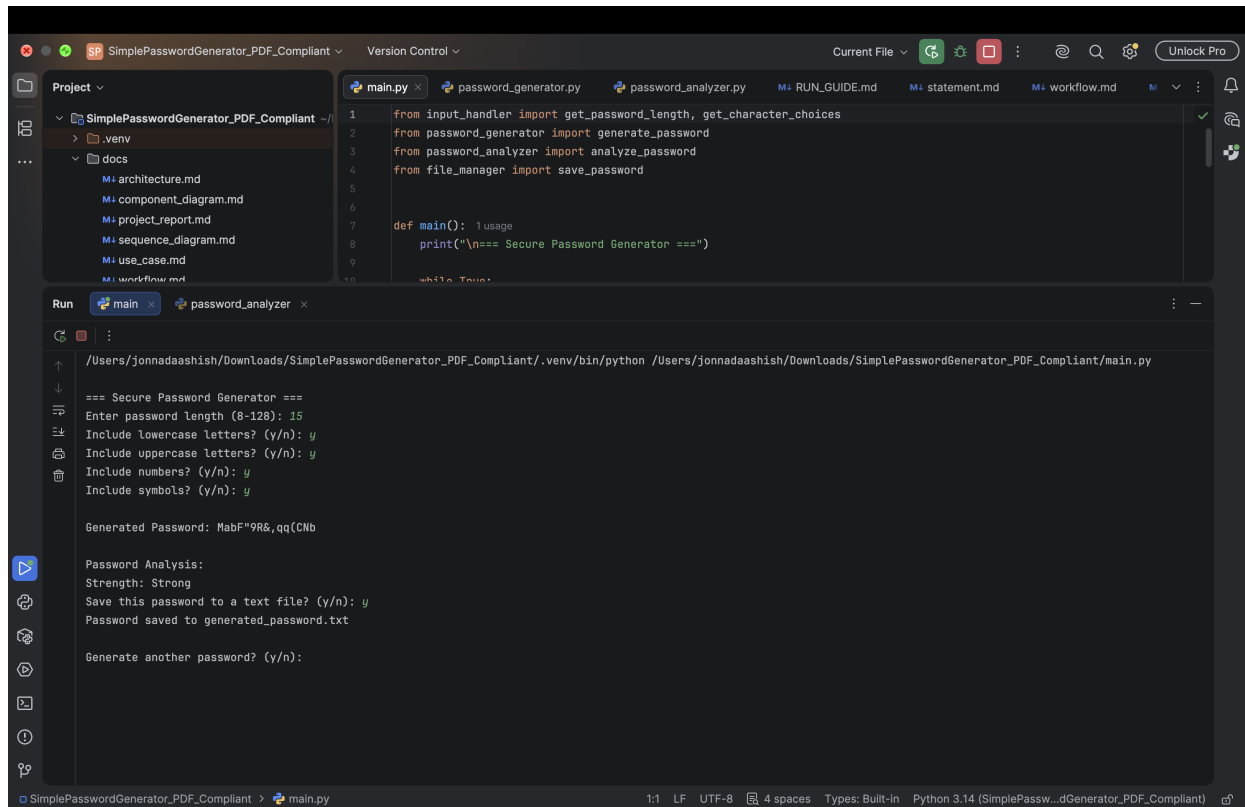
=== Secure Password Generator ===
Enter password length (8-128): 7
Please enter a length between 8 and 128.
Enter password length (8-128): 15
Include lowercase letters? (y/n): y
Include uppercase letters? (y/n): y
Include numbers? (y/n): y
Include symbols? (y/n): n

Generated Password: qybIhS0uIxIs3vm

Password Analysis:
Strength: Strong
Suggestions: Add symbols.
Save this password to a text file? (y/n): n
Password was not saved.

Generate another password? (y/n):
```

Figure 3. Successful password generation with the save option.



10. Testing Approach

Test	Input / Condition	Expected Result
Valid length	16	Accepted
Too short	7	Error and retry
Too long	129	Error and retry
Non-numeric	abc	Error and retry
No categories	All n	Error and retry
All categories	All y	Password generated
Save	y	Text file created
Do not save	n	No file created
Repeat	y	New generation cycle

Testing is manual and scenario-based because the project is small and console-driven. The cases cover valid input, invalid input, character selection, saving, and repeated use.

11. Challenges Faced

- Validating user input without allowing the program to crash.
- Ensuring every selected character category is represented.
- Understanding secure random password generation.
- Keeping the project modular without unnecessary complexity.
- Creating documentation and diagrams that accurately represent a procedural application.

12. Learnings & Key Takeaways

- Learned Python functions, modules, conditions, loops, strings, and dictionaries.
- Learned input validation and error handling.
- Learned secure random generation using the secrets module.
- Practiced basic file handling with pathlib.
- Learned how to divide a program into meaningful modules.
- Learned how to represent software flow using architecture and UML-style diagrams.
- Understood functional versus non-functional requirements.
- Practiced documenting implementation decisions and testing scenarios.
- Learned how to organize a small GitHub-ready software project.
- Learned that a project can remain simple while still having clear modular structure.

13. Future Enhancements

- Add password history with appropriate protection.
- Provide more detailed password-strength analysis.
- Add clipboard support.
- Add configurable password policies.
- Create an optional graphical interface.
- Add automated unit tests.
- Add more examples and user documentation.

14. References

- Python Standard Library documentation.
- Python secrets module documentation.
- Python string module documentation.
- Python pathlib module documentation.
- VITyarthi – Build Your Own Project: General Project Instructions & Submission Guidelines.

END OF PROJECT REPORT